

The Job–Machine Edge Design (`jm_design`) of the BOPO Actor for the Flexible Job Shop Problem

Mariana Correia

October 3, 2026

Abstract

The `jm_design` parameter controls how the job–machine *action edges* of the heterogeneous scheduling graph are built and how the actor scores them. It accepts four values, `auto`, `baseline`, `edges` and `attn` (`JM_DESIGN_CHOICES` in `src/train.py`). This document describes the role of the action edges in the decision process, the three defects of the original (`baseline`) design, how `edges` removes them on the environment side, how `attn` also changes the scoring function of the actor, and what each choice means for the trained model.

Contents

1	Context: where the action edges live	2
1.1	The state graph	2
1.2	From graph to decision	2
2	The four values	3
3	The baseline design and its three defects	3
3.1	How the features are produced	3
3.2	Defect 1: stale features	3
3.3	Defect 2: inconsistent column semantics	3
3.4	Defect 3: one-directional information flow	4
3.5	A worked example	4
4	The edges design	4
4.1	Recomputing every action edge after each decision	4
4.2	The reverse edge type	5
4.3	Effect on the warm-start teacher	5
5	The attn design	5
5.1	Limitation of the linear head	6
5.2	The bilinear term	6
6	Interaction with the batching representations	6
7	Consequences for the trained model	7
8	Usage	7
9	Summary	7

1 Context: where the action edges live

1.1 The state graph

At every decision step the environment (`FJSSPEnv`, `src/env.py`) represents the partial schedule as a heterogeneous graph with three node types: *operation*, *job* and *machine*. Let $\mathcal{J}$ be the set of jobs and $\mathcal{M}$ the set of machines. For each job j let o_j denote its *current* operation, i.e. the first operation of j that has not yet been scheduled, and let $p_{j,m} > 0$ be its processing time on an eligible machine m .

Among the edge types of the graph, one is special:

$$\mathcal{E}_{\text{act}} = \{ (m \rightarrow j) : j \text{ unfinished, } p_{j,m} > 0 \} \quad \text{type } (\text{machine}, \text{exec}, \text{job}).$$

Every edge $(m \rightarrow j) \in \mathcal{E}_{\text{act}}$ is a *candidate action*: “schedule the current operation of job j on machine m ”. Each such edge carries a feature vector $e_{mj} \in \mathbb{R}^5$ (six columns in the batching v2 representations, see Section 6). The `jm_design` parameter only concerns this edge type, its reverse, and the head that scores it.

1.2 From graph to decision

One decision follows the same pipeline under every design:

1. **Mask.** `calculate_mask()` builds the matrix of earliest start times $S \in \mathbb{R}^{|\mathcal{J}| \times |\mathcal{M}|}$ (`job_start_machines`) and the current processing times P (`current_job_proc`). With `mask_option=0` it ranks by start time $S_{j,m}$; otherwise by earliest completion time (ECT) $C_{j,m} = S_{j,m} + P_{j,m}$. For every job, only the `sel_k` best machines stay unmasked.
2. **Normalisation.** `normalize_state()` min-max scales every node-feature column and every edge-feature column to $[-1, 1]$ *independently per column*, over the current graph.
3. **Message passing.** A GNN (GATv2, GINE or TransformerConv), converted to a heterogeneous model by `to_hetero`, computes node embeddings $h_v \in \mathbb{R}^{32}$. `to_hetero` creates one copy of the convolution for each edge type and averages (`aggr='mean'`) the outputs of all relations that point into the same node type.
4. **Scoring.** `ActorModel.forward` gives each action edge a scalar logit s_{mj} . Masked edges get $-\infty$, a softmax turns the logits into a categorical distribution, and one edge is sampled (training) or the argmax is taken (greedy evaluation).
5. **Transition.** `step()` schedules the chosen pair, updates S , P and the machine free times, removes the scheduled job’s action edges, adds new action edges for its next operation, and calls `calculate_mask()` again.

The feature vector e_{mj} is therefore used in two places: inside message passing (the GNN’s edge attention / edge MLP has `edge_dim= 5` inputs) and directly in the score s_{mj} . The quality of e_{mj} matters a lot.

2 The four values

Value	Meaning
auto	Not a design. Resolved at the start of <code>train()</code> by <code>resolve_jm_design</code> : edges for every representation built on <code>ojm</code> (<code>ojm</code> and all <code>ojmb_*</code>), baseline for <code>oo</code> and <code>om</code> . Default of <code>main.py</code> and <code>param.py</code> .
baseline	The original design: action edges in one direction only, with features written once and never updated.
edges	Environment-side fix: reverse edge type added and every action edge recomputed after each decision with one fixed column layout.
attn	The environment of edges , plus a bilinear (attention-style) term in the actor’s scoring head.

Only representations built on `ojm` have job nodes linked to machines. Asking for **edges** or **attn** with `oo/om` raises a `ValueError`. Note that `run_diagnostic_job.py` defaults to **baseline** (not **auto**). This is why the batching sweeps pass `-jm-design edges` explicitly.

3 The baseline design and its three defects

3.1 How the features are produced

Under **baseline** the action-edge features are written *once*, at the moment the edge is created, and there are two different places that create edges:

In `reset()` (first operation of every job; every machine is free and every job is ready at time 0):

$$e_{mj}^{\text{reset}} = \left[p_{j,m}, \frac{p_{j,m}}{\sum_{m'} p_{j,m'}}, \frac{p_{j,m}}{\sum_{m'} p_{j,m'}}, 0, p_{j,m} \right].$$

In `step()` (next operation of the job just scheduled). Let E_j be the end time of job j ’s previous operation, F_m the time machine m becomes free, and $g_{j,m} = p_{j,m} + \max(E_j - F_m, 0)$:

$$e_{mj}^{\text{step}} = \left[g_{j,m}, \frac{p_{j,m}}{\sum_{m'} p_{j,m'}}, p_{j,m} + \max(E_j, F_m), \frac{g_{j,m}}{\sum_{m'} g_{j,m'}}, 0 \right].$$

3.2 Defect 1: stale features

After an action (m^*, j^*) with completion time c^* , the environment updates

$$S_{j,m^*} \leftarrow \max(S_{j,m^*}, c^*) \quad \forall j,$$

so the mask immediately knows that every other job waiting for m^* now starts later. The edge features of those jobs, however, are *not* touched. The network sees the old start/completion values, while the mask uses the new ones. The two contradict each other, and the further the episode goes, the larger the error.

3.3 Defect 2: inconsistent column semantics

Comparing e^{reset} and e^{step} column by column:

Col.	Edge created in <code>reset()</code>	Edge created in <code>step()</code>
0	processing time p	processing time + job wait g
1	relative processing time	relative processing time
2	relative processing time (dup.)	completion time
3	0	relative g
4	processing time p	0

Only column 1 means the same thing in both. The same weight of the network multiplies a ratio for some edges and an absolute time for others, and the column-wise normalisation mixes the two populations. A linear layer cannot learn one consistent use of a column whose meaning depends on when the edge was created.

3.4 Defect 3: one-directional information flow

Only $(m \rightarrow j)$ edges exist. A job receives messages from the machines that can process it, but a machine receives nothing from the jobs that compete for it. Its embedding is built only from the operation-machine relation, so it cannot directly encode “three urgent jobs are waiting for me”.

3.5 A worked example

Two machines M_1, M_2 , two jobs at their first operation: $p_{1,1} = 4$, $p_{1,2} = 6$ (job J_1 is eligible on both) and $p_{2,1} = 3$ (J_2 only on M_1). Use ECT masking with `sel_k`=1.

At reset all starts are 0. The policy schedules J_2 on M_1 , so M_1 is busy until 3 and $S_{1,1} \leftarrow 3$. For J_1 the mask now compares $C_{1,1} = 3 + 4 = 7$ with $C_{1,2} = 0 + 6 = 6$, and keeps M_2 .

	edge $M_1 \rightarrow J_1$	edge $M_2 \rightarrow J_1$
baseline (unchanged since reset)	[4, 0.4, 0.4, 0, 4]	[6, 0.6, 0.6, 0, 6]
edges (refreshed)	[4, 0.4, 3, 7, 0]	[6, 0.6, 0, 6, 0]

Under **baseline**, the network’s features say M_1 is the better machine for J_1 (shorter processing, nothing about the delay), while the mask has already decided that M_2 is better. Under **edges**, column 3 holds exactly the value the mask ranks, and the conflict disappears.

4 The edges design

edges changes *only the environment*. The actor’s architecture is the same as in **baseline**, except that the GNN gets one more relation to process.

4.1 Recomputing every action edge after each decision

At the end of every `calculate_mask()` call (in `reset()` and after every `step()`), `_refresh_jm_edges()` overwrites the features of *all* action edges, masked or not, from the same live matrices the mask uses. With $S_{j,m}$ the earliest start, $P_{j,m} = p_{j,m}$ the current processing time and F_m the machine free time (machine feature 0):

$$e_{mj} = \left[\underbrace{P_{j,m}}_{0: \text{proc. time}}, \underbrace{\frac{P_{j,m}}{\sum_{m'} P_{j,m'}}}_{1: \text{relative speed}}, \underbrace{S_{j,m}}_{2: \text{start}}, \underbrace{S_{j,m} + P_{j,m}}_{3: \text{completion (ECT)}}, \underbrace{\max(S_{j,m} - F_m, 0)}_{4: \text{idle created}} \right].$$

- **Column 0** is the cost of the operation on m .
- **Column 1** compares m with the other eligible machines. A small value means m is a fast choice for this operation.

- **Column 2** is the earliest time the operation can start on m : $S_{j,m} = \max(E_j, F_m)$, kept up to date as other jobs occupy m .
- **Column 3** is the earliest completion time, the very quantity the ECT mask and the warm-start teacher rank by.
- **Column 4** is the gap the machine would sit idle waiting for job j to become ready ($E_j > F_m$). It is a direct signal of wasted capacity.

All five columns mean the same thing for every edge at every step, which fixes Defects 1 and 2.

4.2 The reverse edge type

The same function also writes

$$(\text{job}, \text{exec}, \text{machine}): \quad \text{edge_index} = \text{flip}(\text{edge_index}_{\text{act}}), \quad e_{jm} = e_{mj}.$$

This mirrored relation is used only for message passing; actions are still taken on the $(m \rightarrow j)$ edges. Because `to_hetero` builds one convolution per relation, the machine embedding becomes

$$h_m^{(\ell+1)} = \frac{1}{2} \left(\text{Conv}_{\text{op} \rightarrow m}^{(\ell)}(h^{(\ell)}) + \text{Conv}_{j \rightarrow m}^{(\ell)}(h^{(\ell)}, e) \right),$$

so every machine now aggregates the jobs that compete for it, weighted by the start/completion/idle values of those candidate assignments (Defect 3). With L layers, information can also flow $\text{job} \rightarrow \text{machine} \rightarrow \text{job}$: a job can “see” its competitors through a shared machine.

The reverse edge type is also included in the per-column normalisation of `normalize_state()`.



Figure 1: Action edges (solid, scored by the actor) and the mirrored job $\rightarrow$ machine relation (dashed, message passing only) for the example of Section 3.5.

4.3 Effect on the warm-start teacher

Before BOPO starts, the policy is pre-trained by behaviour cloning on `expert_action()`. Among the unmasked candidates, this rule picks the smallest mask criterion and breaks ties with the other time: start then completion for `mask_option=0`, completion then start otherwise. Ties happen often (71% of decisions with `mask_option=0`). Both quantities are columns 2 and 3 of the refreshed edges, so under `edges` the teacher’s choice is a function of what the network sees, and it can be imitated. Under `baseline` the network sees neither quantity correctly, and the teacher is much harder to fit.

5 The attn design

The environment is identical to `edges`: `FJSSPEnv` treats “edges” and “attn” the same way. The difference is in `ActorModel`.

5.1 Limitation of the linear head

Under **baseline** and **edges** the logit of an action is

$$s_{mj} = w^\top [h_m \parallel e_{mj} \parallel h_j] + b = a^\top h_m + c^\top e_{mj} + d^\top h_j + b.$$

This is *additive* in the machine and the job. For a fixed job j , the term $d^\top h_j$ is the same for every machine. The ranking of j 's machines therefore depends only on $a^\top h_m + c^\top e_{mj}$: the job's own embedding cannot change *which* machine is preferred for it. The head cannot express “this job fits this machine”, only “this job is generally urgent” plus “this machine is generally good”. Any job–machine interaction has to be packed into e_{mj} or created indirectly by message passing.

5.2 The bilinear term

With **attn**, three extra linear maps W_q, W_k, W_e (output dimension $d = 64$) are created, and the logit becomes

$$s_{mj} = \underbrace{w^\top [h_m \parallel e_{mj} \parallel h_j] + b}_{\text{linear head, as before}} + \frac{(W_q h_j)^\top (W_k h_m + W_e e_{mj})}{\sqrt{d}}.$$

This is the same form TransformerConv uses for its attention logits: the job acts as the *query*, the machine (corrected by the edge features) as the *key*. The product $h_j^\top W_q^\top W_k h_m$ is a learned similarity between the two embeddings, so the preferred machine can now differ from job to job. The term $h_j^\top W_q^\top W_e e_{mj}$ lets each job weight the edge features differently (for example, one job cares about idle time, another about completion time). The factor $1/\sqrt{d}$ keeps the variance of the dot product comparable to the linear term at initialisation.

Remark. The extra layers are created only when `jm_design="attn"`. **baseline** and **edges** therefore build exactly the same parameters, and use the random number generator the same way, as before the option existed. Runs with the same seed stay comparable.

6 Interaction with the batching representations

The batching environments (`src/env_batching.py`) override `_refresh_jm_edges()` on top of the base version:

- For the *node* and *edge* batching representations, columns 0 and 3 hold the processing and completion time of the *batch* the action would dispatch (its longest member), since that is what the action really costs. The *base* representation keeps the individual operation's time, so its graph stays free of batching information.
- In batching v2 a sixth column is appended: the number of operations the action dispatches (all zeros for *base*, so that every v2 representation has the same width).
- Wait actions (“wait for a larger batch”) are extra action edges. Only when `jm_design≠baseline` are their columns 2–4 overwritten with the waiting start time s_w , completion $s_w + p$ and idle time $s_w - F_m$. Under **baseline** they would keep the values of the non-waiting action and look identical to it.
- After any edge is added or removed, `_sync_reverse_edges()` mirrors the action edges onto the job→machine relation again.

In short, the batching information is carried mainly by the refreshed action edges. The batching experiments are only meaningful with **edges** (or **attn**), which is why all of them use **edges**.

7 Consequences for the trained model

	baseline	edges	attn
Relations seen by GNN	machine→job only	+ job→machine	+ job→machine
Edge features	stale, two layouts	refreshed each step, one layout	refreshed each step, one layout
Scoring head	linear	linear	linear + bilinear
Extra parameters	—	one conv per layer for the new relation	same as edges + W_q, W_k, W_e
Env. cost per step	lowest	one vectorised re- fresh	same as edges

1. **The architecture depends on the design.** The graph metadata passed to `to_hetero` is read from the first state of the environment (`s.metadata()` in `train()`). Under **edges/attn** that state already contains the reverse relation, so the model has more convolution weights. Under **attn** it also has the three attention maps.
2. **Checkpoints are tied to their design.** A model trained with one value cannot be loaded or evaluated with another. For this reason `jm_design` is stored in `model_params`, and `run_metrics.py`, `rescore_checkpoints.py` and the test code in `train.py` read it back and rebuild the environment with the same value. Old entries without the key are treated as **baseline**.
3. **The features and the mask agree.** With **edges**, the policy sees the quantities that decide whether an action is unmasked and how it ranks. Learning then focuses on choosing *among* the good candidates, instead of first working out a criterion from features that contradict it.
4. **Comparability.** Comparisons between representations or GNN types should keep `jm_design` fixed. Changing it changes both the input and the architecture.

8 Usage

```
# default: auto -> edges for ojm / ojmb_*, baseline for oo / om
python main.py --representation ojm

# explicit choice
python main.py --representation ojm --jm-design baseline
python main.py --representation ojm --gnn-type transformer --jm-design attn

# diagnostic / batching runs (default there is baseline, so set it)
python run_diagnostic_job.py --representation ojmb_node --jm-design edges ...
```

The resolved value is printed at the start of every run:

```
[TRAIN] Representation | ojmb_node_v2 | GNN | gat | jm_design | edges
```

9 Summary

- auto picks **edges** whenever job-machine edges exist, and **baseline** otherwise.

- **baseline**: action-edge features are written once, become stale as the schedule progresses, mean different things depending on where they were created, and flow in one direction only.
- **edges**: every action edge is recomputed after each decision with $[p, p/\sum p, S, S+p, \text{idle}]$, and mirrored onto a job $\rightarrow$ machine relation, so machines see their competing jobs.
- **attn**: **edges** plus a query-key term $(W_q h_j)^\top (W_k h_m + W_e e_{mj})/\sqrt{d}$ that lets the actor score how well a specific job matches a specific machine.